ADVANCED DATA ANALYSIS · MODULE III

Machine Learning II

Advanced models, dimension reduction, and neural networks

M. Fuat Kına

ROADMAP

What this session covers

1 · Advanced machine learning

- Naive Bayes classification
- Support vector machines
- Decision trees and random forests
- Principal component analysis

2 · Introduction to deep learning

- Neural networks and the perceptron
- Forward and back propagation
- Architecture, loss, optimiser
- In practice: scikit-learn and Keras

These are the models you will reach for in Assignment 4 once the baseline is running. The point is not to know all of them — it is to know what each one assumes.

ORIENTATION

Choosing among them

MODEL	TASK	REACH FOR IT WHEN
Naive Bayes	Classification	Text, high-dimensional sparse features; a fast, honest baseline
Support vector machine	Classification	Clear margin between classes; moderate n, many features
Decision tree	Both	Interpretable rules; non-linearity and interactions for free
Random forest	Both	Tabular data where accuracy matters more than a single readable rule
PCA	Unsupervised	Too many correlated features; compression before another model
Neural network (MLP)	Both	Large n, heterogeneous features, patterns no one can hand-specify

None of these is a causal estimate. Week 10 takes that up separately.

1

Advanced Machine Learning

Four workhorses: what each assumes, and where each breaks

Classification by Bayes' theorem

- Supervised learning: a classifier.
- Rests on Bayes' theorem, with one naive assumption — that the features are conditionally independent given the class.
- That assumption is almost always false, and the classifier works anyway. It estimates the ranking of classes well even when the probabilities are poorly calibrated.
- Variants: Gaussian, Multinomial, Complement, Bernoulli — pick by the type of your features.

$$P(\theta \mid data) = P(data \mid \theta) \cdot P(\theta) \,/\, P(data)$$

`P(θ | data)`

Posterior — what we know about the parameters after seeing the data.

`P(θ)`

Prior — what we knew about the parameters beforehand.

`P(data | θ)`

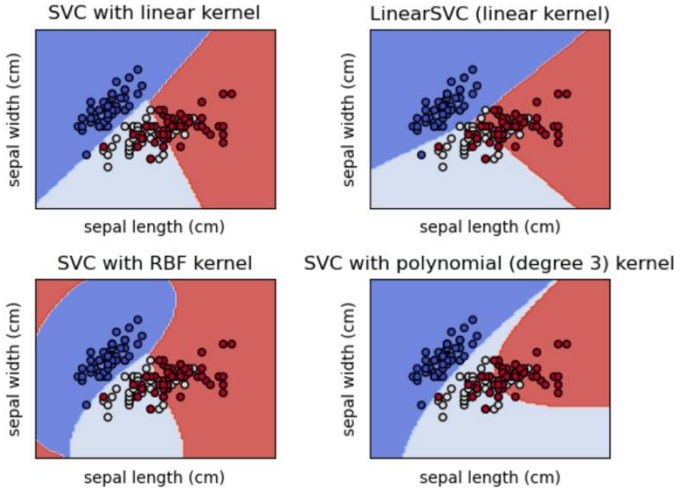
Likelihood — the probability of the data under the statistical model.

```
from sklearn.naive_bayes import MultinomialNB
MultinomialNB().fit(X_counts, y)
```

scikit-learn.org/stable/modules/naive_bayes.html

Maximising the margin

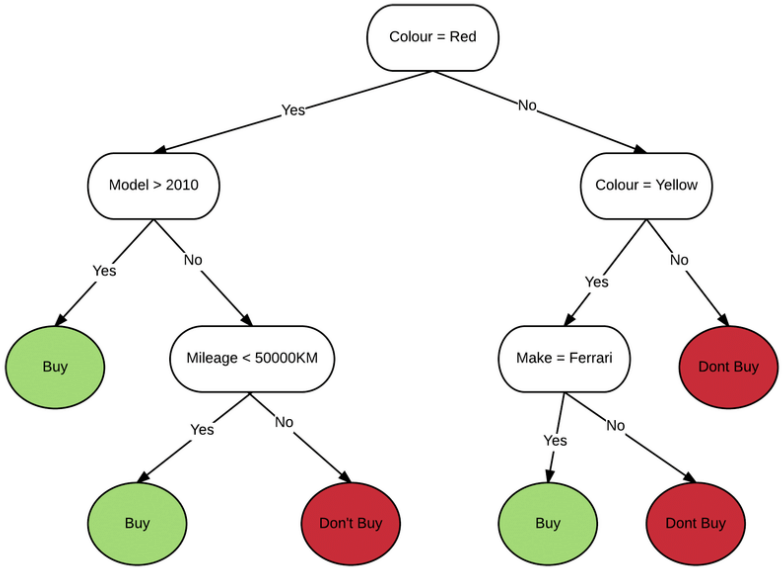
- Supervised learning: a classifier.
- SVM maps the training examples into a space and looks for the separator that maximises the width of the gap between the two categories.
- The gap is measured to the closest points on each side — the support vectors. Everything else could be deleted without changing the boundary.
- Choose a kernel to bend that space: linear, polynomial, RBF.




```
SVC(kernel='rbf', C=1.0, gamma='scale')
```

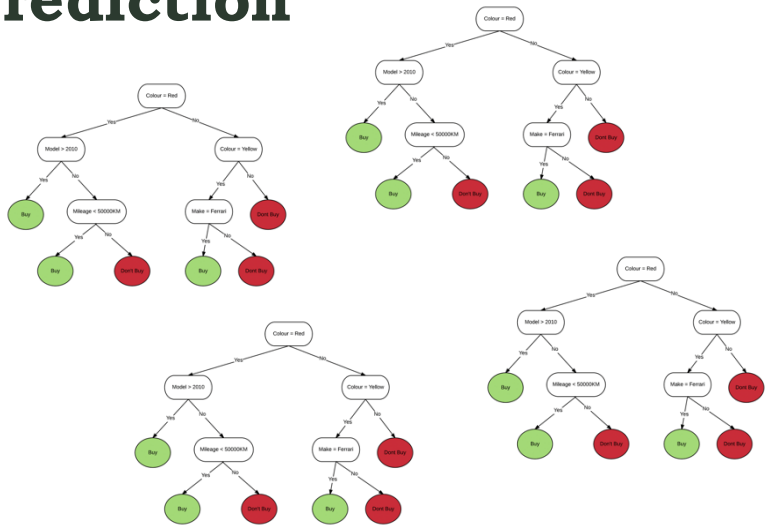
Learning simple rules from the features

- Supervised. The model predicts the target by learning simple decision rules inferred from the features — a piecewise constant approximation.
- Works with categorical and continuous targets alike.
- Split criteria: Gini, entropy, chi-square.
- Overfitting is the default behaviour: a deep enough tree memorises the sample. Cap max_depth and tune it by cross-validation.



Many weak trees, one strong prediction

- One tree is unstable: change a few rows and its structure changes. A forest averages that instability away.
- Each tree is grown on a bootstrap sample, and at each split only a random subset of features is considered — that is what decorrelates the trees.
- The prediction is a vote (classification) or a mean (regression). Specify the number of trees.
- Feature importances come free — and are read too confidently. They reflect predictive contribution, not causal effect.



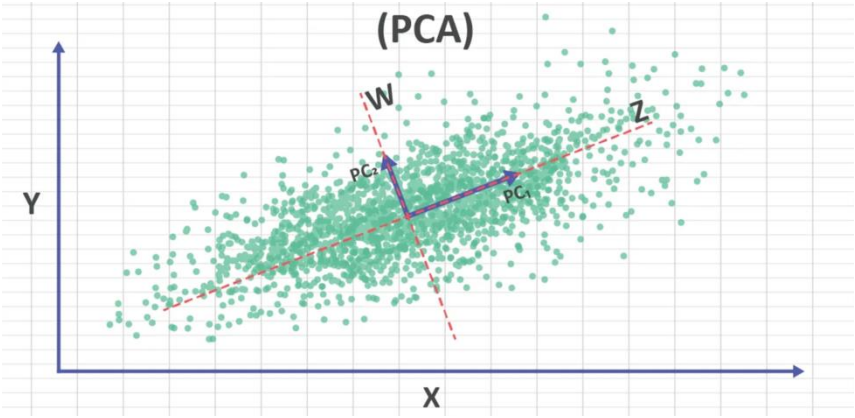
tree 1 tree 2 tree 3 ...

```
RandomForestClassifier(n_estimators=500,
                       max_depth=None,
                       max_features='sqrt',
                       n_jobs=-1, random_state=0)
```

majority vote / average

Fewer dimensions, kept variance

- Unsupervised learning.
- Computes the principal components and uses them to perform a change of basis on the data.
  - The first component is the direction of maximum variance; each next one is orthogonal to the ones before.
- Decorrelates the dimensions: PCA features are uncorrelated with one another by construction.
- Used for compression, for visualisation, and to feed collinear features into a model that cannot handle them.
- The cost: components are linear combinations, so interpretability of the original variables is gone.



2

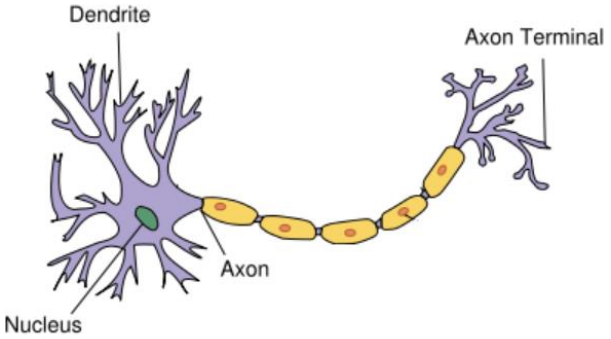
Introduction to Deep Learning

Perceptrons, layers, propagation — and what it costs you

Recognising patterns the way a brain is said to

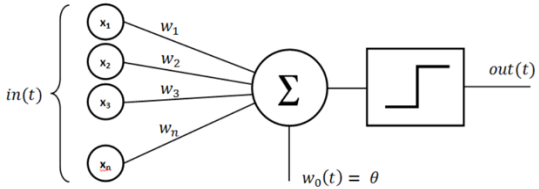
The stated purpose: recognising things the way the human brain does. The inspiration is the nervous system; the resemblance is loose.

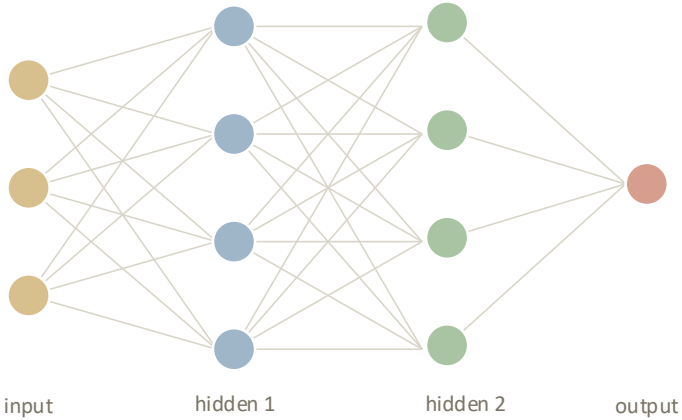
Used within supervised learning.



Recognising patterns the way a brain is said to

- Runs easily in Python with scikit-learn; anything larger goes to Keras or PyTorch.
- Efficient above all when you want to analyse features of different kinds together — text, counts, categories in one model.
- The price: no coefficient to report, a hunger for data, and a result that is hard to defend to a reader who wants a mechanism.





The input layer has as many nodes as the data has features. A single neuron on its own is a perceptron.

HOW IT LEARNS

Forward and back propagation

FORWARD PROPAGATION

- Values entering the input layer are multiplied by weights; a bias is added.
- In the first hidden layer each neuron applies an activation function to what it received.
- Its outputs are multiplied by the next layer's weights — and so on to the output layer.

BACK PROPAGATION

- The error is computed as the difference between the predicted output and the desired one.
- That error is pushed backwards through the layers, and each weight is nudged in the direction that reduces it.
- Repeat over epochs. The whole enterprise is: minimise the error.

One activation function to remember for now: ReLU. It is what makes a deep stack trainable at all.

IN PRACTICE

Architecture, compile, fit, predict

01 **Specify the architecture**

How many layers, and how many nodes in each? Which activation function per layer — identity, ReLU, softmax?

02 **Compile**

Set the loss function (mean squared error; categorical crossentropy) and how optimisation will work (Adam).

03 **Fit the model**

Epochs, batch size, a validation split you actually watch.

04 **Predict the outcome**

And evaluate it with the same metrics as any other classifier — the network gets no exemption.

```
from sklearn.neural_network import
    MLPClassifier

mlp = MLPClassifier(
    hidden_layer_sizes=(64, 32),
    activation='relu',
    alpha=1e-4, max_iter=500,
    early_stopping=True)

mlp.fit(X_tr, y_tr)
mlp.predict(X_te)
```

BEFORE YOU REACH FOR IT

Three cautions for social scientists

01

A baseline first

Penalised logistic regression on the same features, honestly cross-validated. If the network cannot beat it, report the regression.

02

Data, not depth

Most gains in applied social research come from better labels and better features, not from more layers.

03

Report the whole search

Every architecture you tried is part of the analysis. Silently keeping the best run is a specification search with a friendlier name.

Reproducibility applies here too: set every seed, pin your versions, and commit the notebook that produced the number you report.

BEFORE THE NEXT SESSION

What to do with this

Lab	Run Naive Bayes, SVM, a forest and an MLP on the same split, and put the scores in one table.
Read	VanderPlas, Chapter 5 (second half); Jurafsky & Martin, chapters 9–11.
Assignment 4	The shared task is live. Compare models honestly; the pipeline is the deliverable, not the best number.
Next	Causal inference and machine learning — why a good predictor is not a causal estimate.

Credits: VanderPlas, Python Data Science Handbook · scikit-learn documentation · Keras guides